

ARIA

Complete Project Guide

ARIA (Agentic Real-estate Intelligence Advisor) is a multi-agent artificial intelligence system for short-term rental investment intelligence, covering the Paris and Athens markets. This guide explains the entire project — the data, every model, the architecture, and the live product — in precise, plain language written for every member of the team.

135,051 listings analysed

Paris + Athens

7 phases · all complete

Live demo on Vercel

5 specialist agents

How to read this document. Every technical term is explained the first time it appears in each section, and every abbreviation is followed by its full name in brackets — for example, RAG (Retrieval-Augmented Generation). A complete alphabetical glossary closes the document, so any page can also be read on its own.

What this guide covers

01	Executive summary	3	10	Phase 6 — LangGraph orchestration	22
02	The problem ARIA solves & who it serves	5	11	Phase 7 — The live product	24
03	The data foundation	6	12	Business findings & recommendations	26
04	System architecture	9	13	Trusted AI & governance	28
05	Phase 1 — Exploratory Data Analysis	12	14	Risks & limitations	29
06	Phase 2 — XGBoost price prediction	13	15	Mentor Q&A preparation	31
07	Phase 3 — LightGBM host-risk model	16	16	Glossary of every technical term	34
08	Phase 4 — Prophet demand forecasting	19	17	Repository map & credits	36
09	Phase 5 — RAG compliance agent	20			

Sources for this guide. Everything here is drawn from the repository itself: the README, the model card, the six analytical notebooks, the mentor-preparation documents, and the deployed application code. Where a number appears — a score, a count, a euro value — it is the number recorded in the committed project files, not an estimate.

What ARIA is, in one page

ARIA (Agentic Real-estate Intelligence Advisor) answers one family of questions: **where is the opportunity in a short-term rental market, and what exactly should a decision-maker do about it?** It does this by combining validated machine learning models with a conversational product that explains every recommendation in business language.

The project was built in seven phases and is best understood as **three layers working together**:

1 · The research and modelling layer (notebooks)

Six Jupyter notebooks (interactive documents that mix code, results, and explanation) prove the analytical methods: exploratory data analysis, price prediction, host-risk classification, demand forecasting, regulation retrieval, and multi-agent orchestration. Each notebook was graded against an explicit rubric and each documents its own limitations.

2 · The evidence layer (committed model outputs)

Every model writes its results to version-controlled files — prediction tables, risk scores, forecast tables, compliance reports. These files are the **single source of truth**: the live product is only allowed to quote numbers that exist in them. This is what makes ARIA's answers verifiable rather than improvised.

3 · The live agent layer (deployed web product)

A chat interface, deployed on Vercel (a cloud hosting platform), turns those outputs into plain-English recommendations with KPI (Key Performance Indicator) cards, charts, interactive maps, and exportable briefs. A user asks a question in natural language; ARIA routes it to the right specialist, grounds the answer in the committed evidence, and explains the reasoning.

The one-sentence story: ARIA does not simply chat about real estate — it routes each question to the right analytical agent, grounds the answer in validated project outputs, visualises the market signal, and explains the recommendation so a non-technical reader can act on it.

The headline numbers

These are the figures the whole project stands on. Each one is explained in depth in its own section; every value comes from the committed output files.

135,051

Listings in the master dataset

96 columns per listing · Paris
120,809 · Athens 14,242

594,482

Guest reviews sentiment-scored

Scored with the VADER sentiment
method during Phase 1

7 / 7

Project phases complete

From data analysis to a live deployed
product

2,945

Underpriced Athens listings

Priced more than €15 below the
model's fair price

865

Priority listings — the hero output

Underpriced AND high-risk at the
same time

€4.8M

Foregone revenue per year

Estimated annual revenue left on the
table in Athens

0.676

Athens pricing accuracy (R^2)

Share of price variation the model
explains — target was 0.65

0.8288

Risk model quality (AUC)

Honest score after finding and fixing
data leakage

137

Unlicensed Athens listings

≈ €1.03M annual revenue exposed to
compliance action

Why these numbers matter together

Each number alone is an observation. Together they form a decision pipeline: the dataset gives breadth, the pricing model finds money left on the table, the risk model finds listings in trouble, their **865-listing overlap** identifies where pricing improvement and risk intervention matter simultaneously, and the compliance work identifies where regulation changes the picture entirely. That chain — from raw data to a ranked, explainable action list — is the project's core achievement.

A note on honesty: the risk model's first version scored a suspiciously perfect 0.9995. The team investigated, found data leakage (the model accidentally seeing the answer), removed it, and reported the honest 0.8288. That correction story is a feature of this project, not an embarrassment — it is exactly what KPMG's Trusted AI principles ask for.

Three users, one advisory engine

Short-term rental decisions are usually made on partial evidence: a price spreadsheet here, a gut feeling about a neighbourhood there. ARIA's premise is that these decisions should combine **pricing opportunity, host risk, future demand, geography, and regulation** — and that the reasoning should be explainable to a non-specialist.

● Institutional investor

"Which neighbourhood should I enter, what is the yield, and what is the downside risk?"

ARIA ranks neighbourhoods by an opportunity score, shows median nightly prices and revenue signals on a map, and pairs every recommendation with the saturation and risk picture, so entry decisions weigh upside against crowding.

● Host / property manager

"Am I priced correctly, is my listing declining, and what exactly should I improve?"

ARIA compares a listing's actual price with the model's fair-price estimate, flags the gap, and uses the model's own explanation values to suggest concrete moves — adjust the price by a specific amount, or improve the specific feature dragging the prediction down.

● Real-estate developer / private-equity fund

"Where is the supply-shock opportunity and what is the realistic entry price at scale?"

ARIA frames markets at portfolio level: listing volumes, saturation pressure, compliance exposure (for example, the 137 unlicensed Athens listings whose revenue could exit the market), and cross-city comparison between Paris and Athens.

Why Paris and Athens

Paris provides **scale** — 120,809 listings across two vintages of data — and proves the framework works in a large, complex market. Athens provides **currency and completeness** — 14,242 listings with observed September 2025 prices, licence information, review momentum, and calendar-derived signals. Athens is therefore the main proving ground for actionable recommendations, while Paris serves as the comparison market. This split is a deliberate, documented decision, not an accident of the data.

Four public sources, one master dataset

Every model and every answer in ARIA depends on one asset: a master analytical dataset of **135,051 listings × 96 columns**, assembled from four public sources. No single source had everything the project needed — prices, locations, reviews, availability, host behaviour, licences, and neighbourhood context — so they were combined.

SOURCE	CITY	LISTINGS	WHAT IT CONTRIBUTED
Maven Analytics (2021)	Paris	63,520	A large Paris base with observed prices and rich host and listing attributes.
Inside Airbnb (Sept 2025)	Paris	57,289	A current view of Paris demand, availability, and host behaviour. Its price field was partly blank and had to be estimated — which is why these rows are excluded from price training.
Inside Airbnb (Sept 2025)	Athens	14,242	Current Athens prices, licences, reviews, availability, and calendar-derived signals — the most complete and current slice of the dataset.
Gyodi & Nawaro (2021)	Both	enrichment	Spatial context from a published academic dataset: attraction, restaurant, and metro-distance indices joined at zone level.

The idea that protects every model: routing flags

Because the sources differ in quality and vintage, the team engineered **eligibility flags** — columns that mark which rows are safe for which task. This is one of the quietly most important decisions in the project:

FLAG COLUMN	PLAIN-LANGUAGE MEANING
price_eur_estimated	This listing's price was estimated, not observed — descriptive use only.
xgb_price_training_eligible	Safe to use for training the price model (observed price present).
prophet_training_eligible	Suitable for time-series demand forecasting.
sentiment_city_comparable	Sentiment score reliable enough for cross-city comparison.

Why this matters: training a pricing model on estimated prices would teach it the estimation method instead of the real market. The flags make that mistake structurally impossible — inappropriate rows never reach the model.

The engineering behind the dataset

“Combining four sources” sounds simple; in practice it required 41 documented pipeline steps. The most consequential ones, in plain language:

Encoding and format repair

The source files used different text encodings (the byte-level way characters are stored) and different formats for the same ideas — true/false fields, currencies, dates. All were standardised before any analysis, so the models never see formatting artefacts.

Price normalisation and the log transform

Prices arrived in different formats and units. Once normalised into euros, prices were also transformed into **log_price** (the logarithm of price) for modelling. Rental prices are heavily skewed — most listings are moderately priced, a few are extremely expensive. Predicting the logarithm compresses those extremes so the model learns the whole market instead of obsessing over luxury outliers. Predictions are converted back to euros before anyone sees them.

Geography: the Haversine distance

Each listing's distance to the city centre was computed with the **Haversine formula**, which converts two latitude/longitude points into the distance between them along the Earth's surface. Listings were then grouped into distance zones — centre, near, mid, far — because location proved to be one of the strongest price drivers.

Sentiment: 594,482 reviews scored

Guest reviews were scored with **VADER** (Valence Aware Dictionary and sEntiment Reasoner), a rule-based method that rates text from very negative to very positive. One honest finding: VADER reads French less reliably than English, inflating Paris scores by about +0.178 — so cross-city sentiment comparison is explicitly ruled out and marked by a flag.

The engineered risk label: at_risk_host

Public Airbnb data contains no official “this host is failing” label. The team therefore engineered a transparent proxy from **six observable warning signs** — recent review momentum below neighbourhood peers, more than 200 available nights per year, declining year-over-year review growth, guest rating below 8.0 out of 10, host tenure under one year, and absence of the Superhost quality badge. A listing showing **three or more** signs is labelled at-risk. The label is deliberately broad: it is an early-warning signal for prioritisation, not a verdict.

What the data revealed before any model ran

Exploratory analysis produced findings that shaped the whole system design:

2.8×

Athens distance effect vs Paris

Distance to the centre moves Athens prices 2.8 times more strongly — the key justification for training separate city models

+0.178

VADER French-language bias

Paris sentiment scores inflated by the method's weaker French reading — cross-city sentiment comparison ruled invalid

137

Unlicensed Athens listings

Reframed from a data-quality footnote into a compliance and supply-shock use case

3.9×

Centre-vs-far revenue premium

Athens centre ≈ €7,236/yr vs far ≈ €1,848/yr estimated revenue

2.4%

Best gross yield zone: "Near"

Centre 2.3% · Near 2.4% (best entry) · Mid 1.8% · Far 2.1%

0.955

Top opportunity score: Zappeio

The Zappeio area (ΖΑΠΠΕΙΟ) leads Athens with ≈ €15,416/yr estimated revenue

How to read these findings

The 2.8× distance effect means the two markets behave differently at a structural level: Athens tourism concentrates around the Acropolis and Syntagma Square, while Paris demand spreads across many districts. One combined pricing model would average these behaviours away — so ARIA trains one model per city.

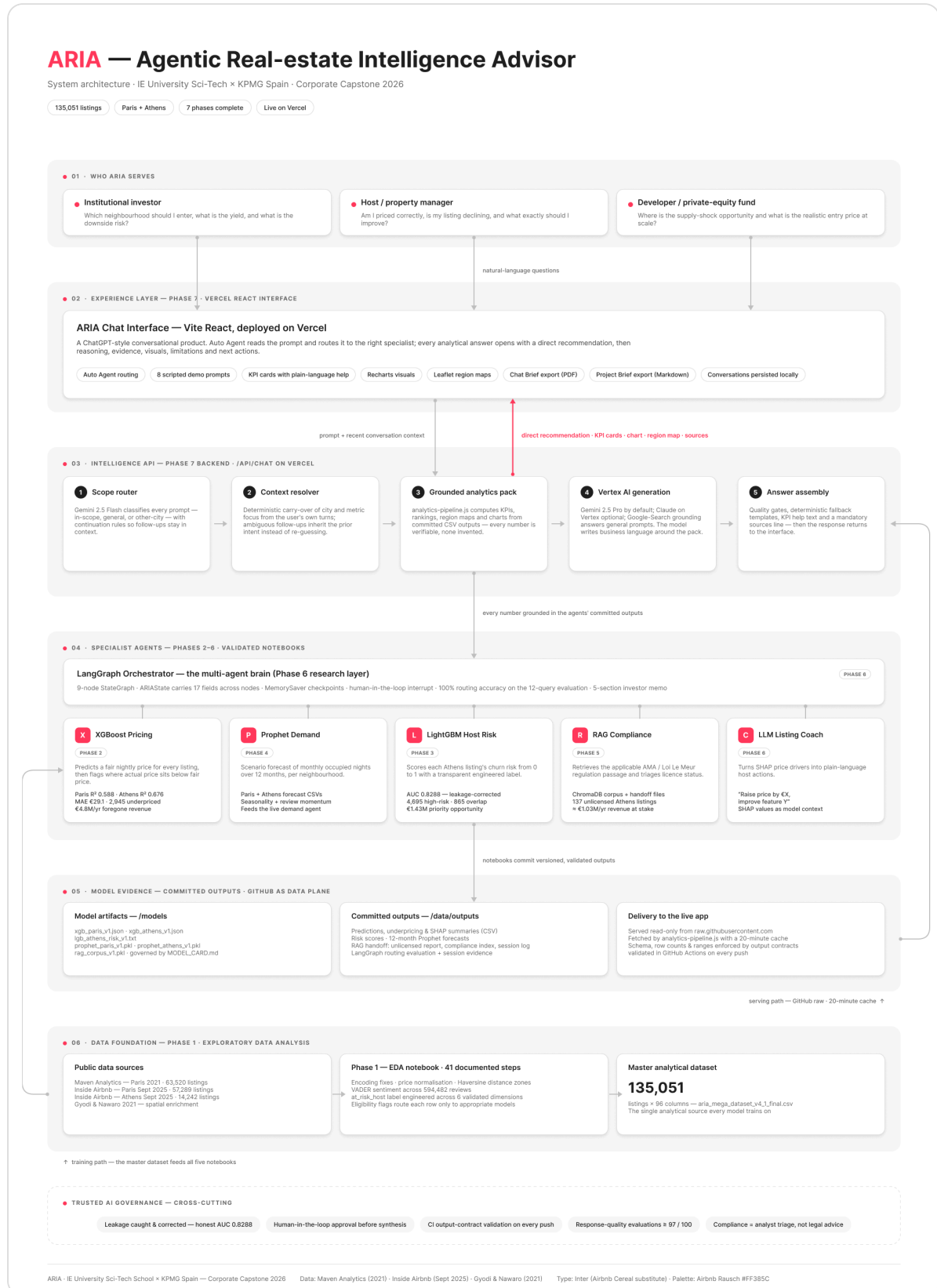
The yield ladder (gross yield is annual rental revenue divided by property price, before costs) says the smartest Athens entry is often not the postcard centre: the "near" ring pairs strong revenue with lower purchase prices.

Zappeio's 0.955 opportunity score — a 0-to-1 composite of revenue upside and market pressure — is the single strongest neighbourhood signal in the dataset, and it is the answer the live product leads with when an investor asks where to start in Athens.

Data honesty rule inherited by everything downstream: Paris pricing conclusions are always presented with the caveat that part of the Paris data is from 2021 and part of the 2025 price field was estimated. Athens carries the current-market story.

The whole system on one canvas

Designed in Figma using Airbnb's design language — the same visual system the live product's default theme follows. Read it top to bottom: people ask, the experience layer converses, the intelligence API grounds, the agents validate, the evidence stores, the data founds. The red path is the answer travelling back to the user.



Layer by layer, in plain language

Layer 01 — Who ARIA serves

The three personas from section 02. Everything below exists to answer their questions.

Layer 02 — Experience layer (Phase 7 · the chat interface)

A React web application (React is a widely used framework for building interactive websites) deployed on Vercel. It offers Auto Agent routing, eight scripted demo prompts, KPI cards, charts, interactive maps, and one-click export of conversation briefs. It holds no analytics logic itself — it displays what the layer below returns.

Layer 03 — Intelligence API (Phase 7 · the backend)

The server-side brain of the live product, running as an API (Application Programming Interface — code that other software talks to) on Vercel. Five steps run for every question: a **scope router** decides whether the question is in ARIA's domain; a **context resolver** carries the city and metric focus across follow-up questions; the **grounded analytics pack** computes every number, chart, and map from the committed outputs; **Vertex AI generation** (Google Cloud's model platform, running Gemini 2.5 Pro by default with Anthropic's Claude as an option) writes the business explanation around those verified numbers; and **answer assembly** applies quality gates and attaches sources before anything reaches the user.

Layer 04 — Specialist agents (Phases 2–6 · the validated intelligence)

The five analytical specialists — XGBoost pricing, Prophet demand, LightGBM risk, RAG compliance, and the LLM (Large Language Model) listing coach — plus the LangGraph orchestrator that wires them into one graph with human approval built in. Sections 06–10 cover each in depth.

Layer 05 — Model evidence (the data plane)

Every validated output is committed to the repository and served to the live app read-only via GitHub's raw file service, fetched with a 20-minute cache. A CI (Continuous Integration — automated checks that run on every code push) contract validates schemas, row counts, and value ranges, so the app can never silently consume malformed evidence.

Layer 06 — Data foundation (Phase 1)

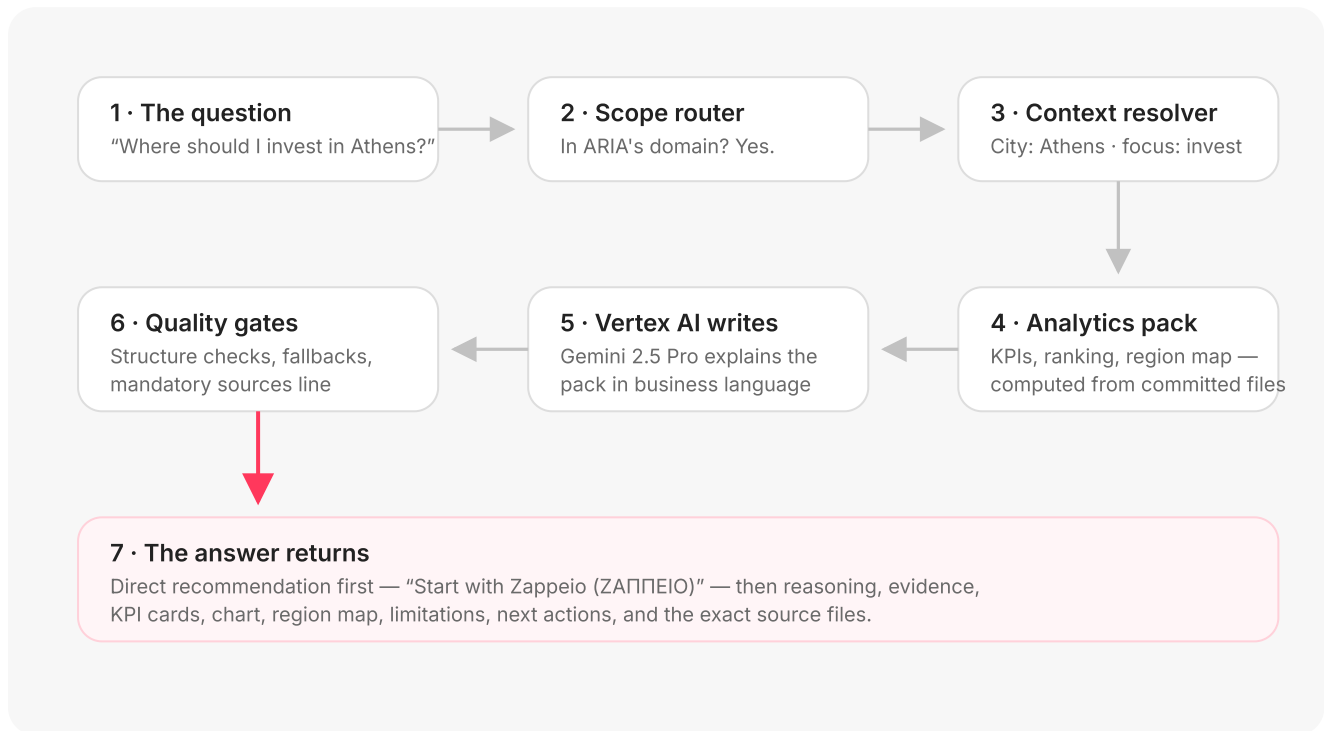
The four public sources and the 135,051 × 96 master dataset described in section 03.

The two vertical paths on the diagram's edges

Training path (left): the master dataset feeds all five modelling notebooks. **Serving path (right):** committed outputs flow up into the live API. Notebooks touch the app only through versioned, validated files — never directly.

The journey of one question

What actually happens in the seconds after a user presses Enter:



Two rules that define the system

Rule 1 — The language model never invents numbers. Step 4 computes every figure deterministically from the committed files *before* the language model is called. The model's job is language, not arithmetic. If its answer arrives incomplete or unstructured, step 6 substitutes a deterministic template built from the same pack.

Rule 2 — Context carries forward deliberately. Follow-up questions like “now show me the map” inherit the city and the metric focus from the user's own earlier turns, so ARIA's second answer stays consistent with its first instead of re-guessing.

The phase that made every other phase defensible

EDA (Exploratory Data Analysis) means systematically inspecting a dataset before modelling: what exists, what is missing, what is unreliable, and what patterns should shape the design. Phase 1 ran **41 documented steps** across the 135,051 listings and was graded A+ (99/100) against the project rubric.

What it actually did

- **Audited every source** — encodings, missing values, duplicated meanings, and price reliability, producing the routing flags described in section 03.
- **Engineered the features** the models would later use: distance zones, interaction terms, review-momentum measures, host-professionalism signals, and the composite review score rebuilt onto a comparable 0–10 scale across sources.
- **Created the `at_risk_host` label** across six validated dimensions — the foundation of Phase 3.
- **Documented limitations honestly** — the Paris vintage problem, the VADER language bias, the absence of a true churn label — so downstream phases inherit disclosed, managed weaknesses rather than hidden ones.
- **Produced 40 figures** used across project documents and the defense.

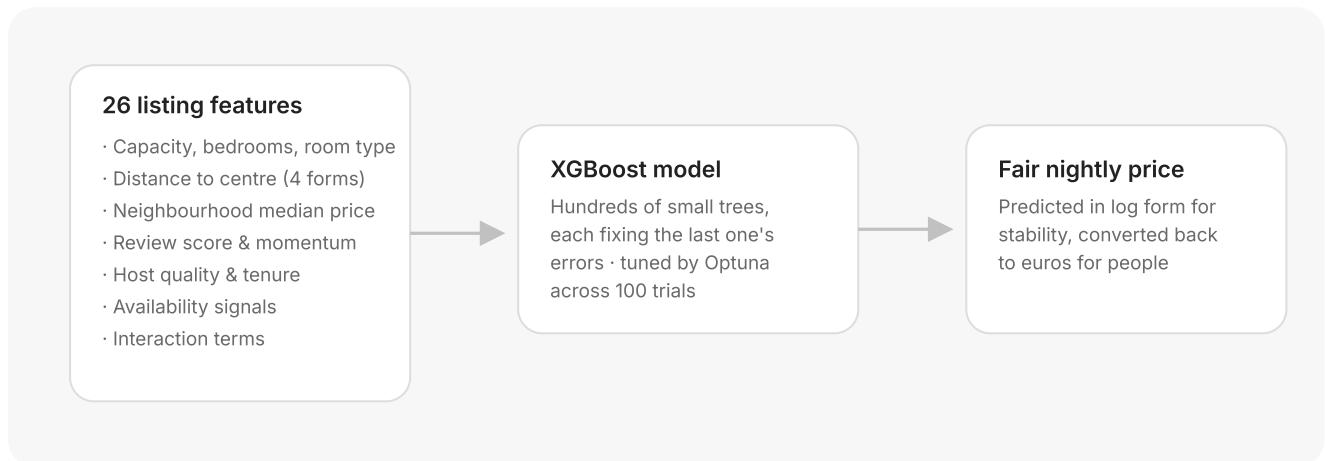
Why so much effort before any model

Every mentor-facing claim in this project traces back to a Phase 1 decision. Why separate city models? Because EDA measured the 2.8× distance-effect difference. Why log-price? Because EDA measured the skew. Why is Athens the hero market? Because EDA proved its data currency and completeness. Why exclude certain columns from the risk model? Because EDA mapped which columns could leak the label. A model is only as defensible as the analysis that justified it.

Output: the master dataset `aria_mega_dataset_v4_1_final.csv` (135,051 rows × 96 columns), saved to `data/processed/` and consumed by every subsequent phase. Large raw files stay out of version control by policy; the notebook regenerates the master dataset end-to-end.

Teaching a model what “fair price” means

XGBoost (Extreme Gradient Boosting) is a machine learning method that builds hundreds of small decision trees, each one correcting the errors of the previous ones. It excels at exactly this kind of problem: structured table data where price depends on **combinations** of factors rather than any single one.



Design decisions that matter

One model per city. Athens prices react to distance 2.8× more strongly than Paris prices; the model explanations later confirmed that Paris pricing is size-and-capacity driven while Athens is quality-location-demand driven. One combined model would blur both.

Observed prices only. Training uses only rows flagged `xgb_price_training_eligible` — listings whose price was actually observed. Estimated prices never become training labels.

Honest evaluation. 80% of eligible rows train the model; a locked-away 20% holdout measures it. Optuna (an automated tuning tool) searched 100 configurations of tree depth, learning rate, and regularisation. Paris was split preserving distance-zone balance; Athens used a random split because its far zone (53 listings) is too small to stratify reliably.

How good is it — and how to say so precisely

0.588

Paris R^2 (R-squared)
Above the 0.52–0.58
range published for
comparable 2021-
vintage data

0.676

Athens R^2 (R-squared)
Beat the project target of
0.65 — the main
actionable model

€29.1

Mean absolute error
(both cities)
Average size of the
model's pricing mistake
per night

+44%

Athens improvement vs
naive
Versus always guessing
the average price (+36%
for Paris)

What these metrics mean, exactly

R^2 (R-squared, the coefficient of determination) is the share of price variation the model explains, from 0 (explains nothing) to 1 (explains everything). Athens 0.676 means the model captures about two-thirds of why prices differ between listings — strong for public data without photos or exact addresses.

MAE (Mean Absolute Error) is the average size of the prediction error in euros. €29.1 means a typical prediction lands within about €29 of the true nightly price. The practical consequence: small predicted gaps must be treated as hints, and only larger gaps as confident signals — a rule the underpricing logic on the next page applies explicitly.

Explainability: SHAP

SHAP (Shapley Additive Explanations) assigns each feature a signed contribution to each individual prediction — “the neighbourhood pushed this price up €18; the low review score pulled it down €6.” The project exports SHAP summaries for both cities, and they power two things: the mentor-facing evidence of *why* the model prices as it does, and the listing coach agent (section 10) that turns drivers into host actions. SHAP is also the tool that later caught the risk model's leakage — explainability doubling as quality control.

Feature honesty: the model never sees the listing's own price or anything derived from it. The neighbourhood context comes from `neighbourhood_median_price`, a market benchmark — a listing is judged against its local market, never against itself.

From predictions to money: the underpricing gap

Once the model knows a fair price for every listing, one subtraction creates the business product: **gap = predicted fair price – actual price**. A large positive gap means the listing is probably charging less than the market would bear.

One Athens listing, illustrated

Actual price

€60

gap = €25

Model's fair price

€85

Flagged underpriced when the gap exceeds €15 — a discovery threshold, deliberately below the €29.1 average model error. Gaps above €25–€40 are the high-confidence tier; the €25 median gap across all 2,945 flagged listings sits exactly at that confidence boundary.

2,945

Athens listings flagged

Predicted fair price more than €15
above actual price

€25

Median gap per flagged listing

Half the flagged listings have a
larger gap than this

€4.8M

Foregone revenue per year

Sum of gaps × expected booked
nights across flagged listings

How to defend the €15 threshold

It is a **discovery threshold, not a pricing command**. With an average model error of €29.1, a €16 gap alone does not prove underpricing — it nominates a candidate for review. ARIA therefore ranks candidates by gap size, treats gaps above €25–€40 as higher-confidence, and always pairs the number with its SHAP explanation. The €4.8M figure is the aggregate over all candidates and should be quoted as an opportunity estimate, not a guarantee.

Output files: `paris_predictions_v1.csv` · `athens_predictions_v1.csv` · `athens_underpricing_v1.csv` · `shap_paris_v1.csv` · `shap_athens_v1.csv` — plus the two trained models in `models/` .

A second model for a different question: risk

The pricing model asks “what should this cost?”. The risk model asks “is this listing in trouble?”. LightGBM (Light Gradient Boosting Machine) is a fast tree-based classifier — instead of a euro value, it outputs a **probability from 0 to 1** that a listing matches the at-risk profile engineered in Phase 1.

The risk probability scale, with Athens listing counts

Low · 0.00–0.40

6,137 listings

Moderate · 0.40–0.70

3,410 listings

High · 0.70–1.00

4,695 listings — the watchlist

The label, restated precisely

No public dataset says “this host churned”, so the target is the engineered **at_risk_host** label: three or more of six observable warning signs (weak recent review momentum vs neighbourhood peers, more than 200 open nights per year, declining year-over-year review growth, rating below 8.0/10, host tenure under one year, no Superhost badge). The class balance is 56.8% at-risk vs 43.2% not — moderately imbalanced, handled by the model's built-in class weighting rather than synthetic data.

Eleven features, chosen to avoid self-deception

The final model uses **11 features** — review momentum and growth, composite review score, monthly review volume, host professionalism (multi-listing, tenure, Superhost), amenity depth, distance to centre, capacity, and room type. Just as important is what was **excluded**: any column used to construct the label itself (availability counts, the normalised rating) and anything derived from price or occupancy outcomes. Those exclusions are the leakage lesson on the next page.

Why Athens only: the label depends on current availability, review momentum, and calendar-derived behaviour. Only Athens has all of these in current (September 2025) form — Paris lacks a current calendar file and has the price-vintage problem.

The leakage story — the project's best governance moment

The first version of this model scored an AUC of **0.9995** — near-perfect. In machine learning, near-perfect on messy public data is not good news; it is a symptom. The team treated it as one.

1. **Suspicion.** A score that high implies the model is not predicting — it is reading the answer from somewhere.
2. **Diagnosis with SHAP.** The explanation values showed one feature, **availability_pressure**, dominating every prediction. That column is computed from the same availability signal used to build the risk label itself — the model had been handed the answer key. This is called **data leakage**.
3. **Correction.** Every label-derived or outcome-derived column was removed (availability counts, normalised rating, occupancy and revenue estimates, prices).
4. **Honest re-measurement.** The corrected model scores AUC **0.8288** — genuinely strong discriminative power, honestly earned, and stable across validation folds.

0.8288

AUC-ROC · target > 0.72 ✓

Probability the model ranks a random risky listing above a random safe one

0.8864

Average precision · target > 0.65 ✓

Quality of the ranked at-risk shortlist — what a reviewer works through

0.1656

Brier score · target < 0.25 ✓

Calibration: predicted probabilities behave like real probabilities

±0.0088

Cross-validation stability ✓

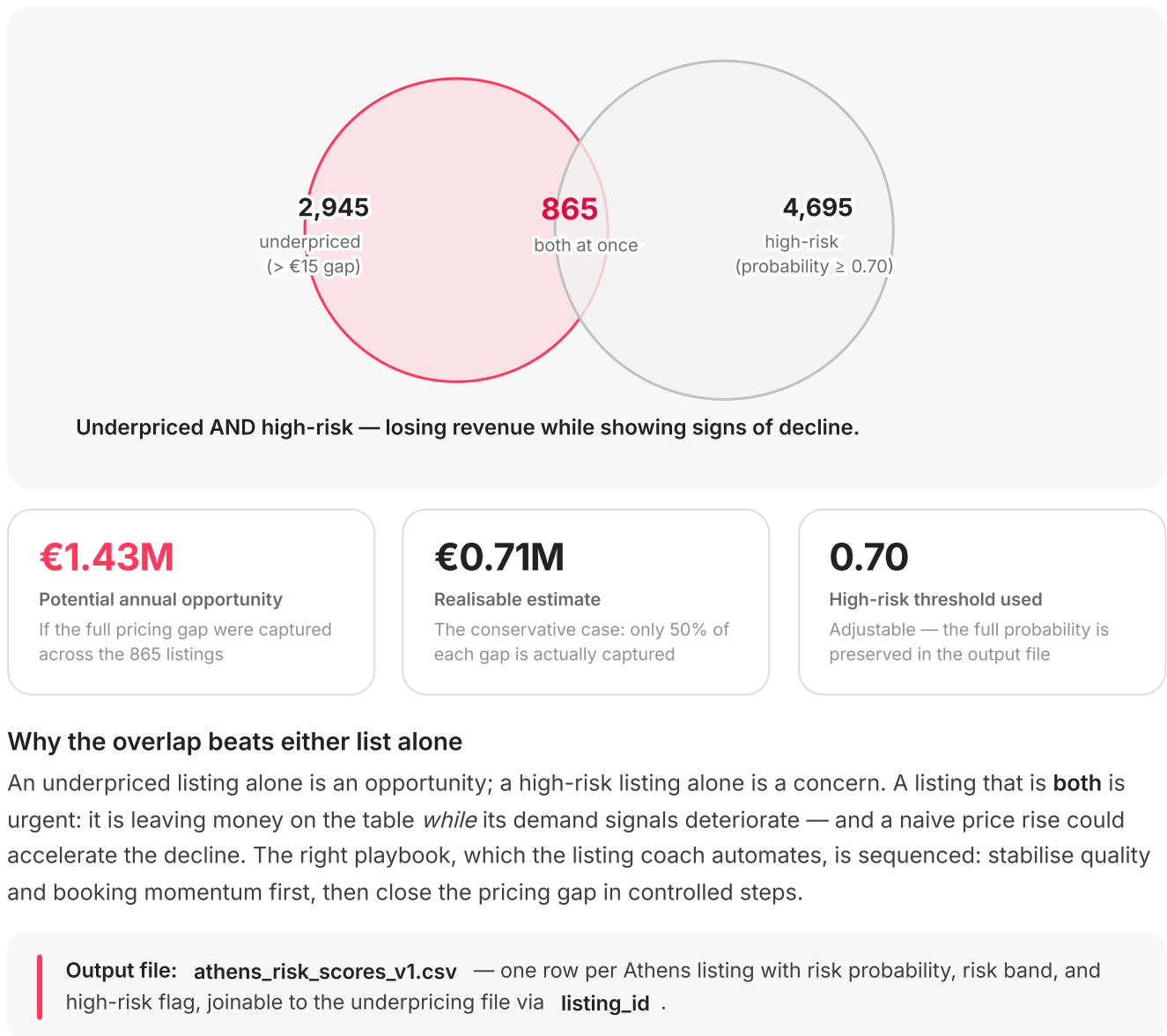
Performance varies less than ±0.01 across five data splits

The metrics, in one sentence each

AUC-ROC (Area Under the Receiver Operating Characteristic Curve) measures ranking quality across all thresholds — the right lens for a prioritisation tool, and immune to the trap where a lazy model scores 56.8% “accuracy” by calling everything risky. **Average precision** focuses on how clean the top of the ranked list is. **Brier score** checks the probabilities themselves are trustworthy. **Cross-validation** re-trains the model on five different splits to prove the result is not luck.

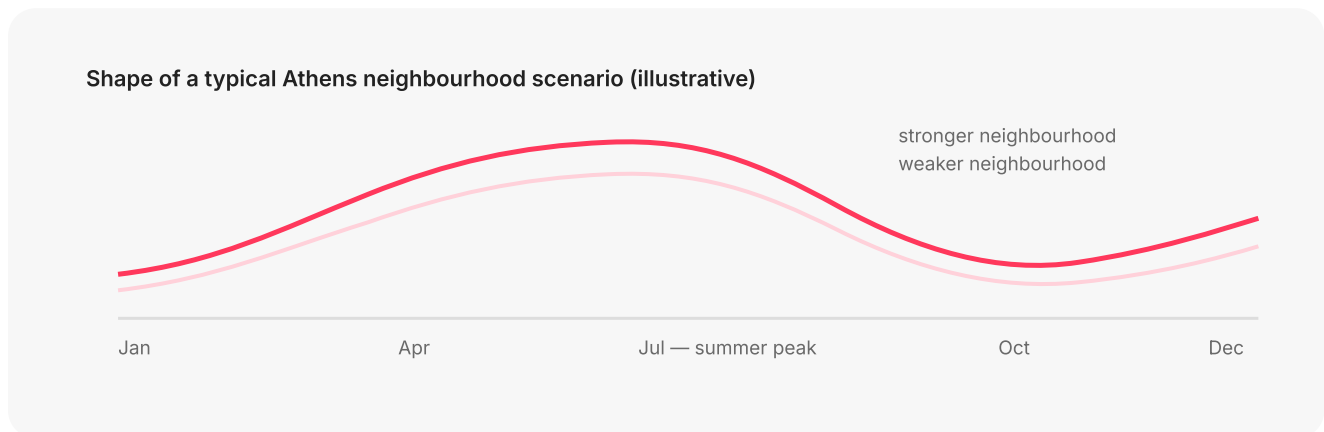
The hero output: 865 priority listings

The project's single most actionable finding comes from joining the two models on the listing identifier:



Adding time: what happens to demand next year?

Pricing and risk describe the present. **Prophet** — a time-series forecasting method developed at Facebook, designed for business data with strong seasonality — adds the future: a **12-month, neighbourhood-level scenario forecast of monthly occupied nights** for both cities.



What “scenario forecast” means — the honest framing

The committed forecasts are **scenario-based demand proxies**, built from annual occupancy estimates, assumed short-term-rental seasonality, and capped review-growth momentum. They are **not** trained on observed booking calendars (which public data does not provide), and the notebook records that forecast intervals fell below the coverage target. The correct use — and the only use ARIA makes of them — is as a **demand screen**: compare neighbourhoods' relative demand shape and timing before property-level due diligence, rather than treating any single month's number as a booking guarantee.

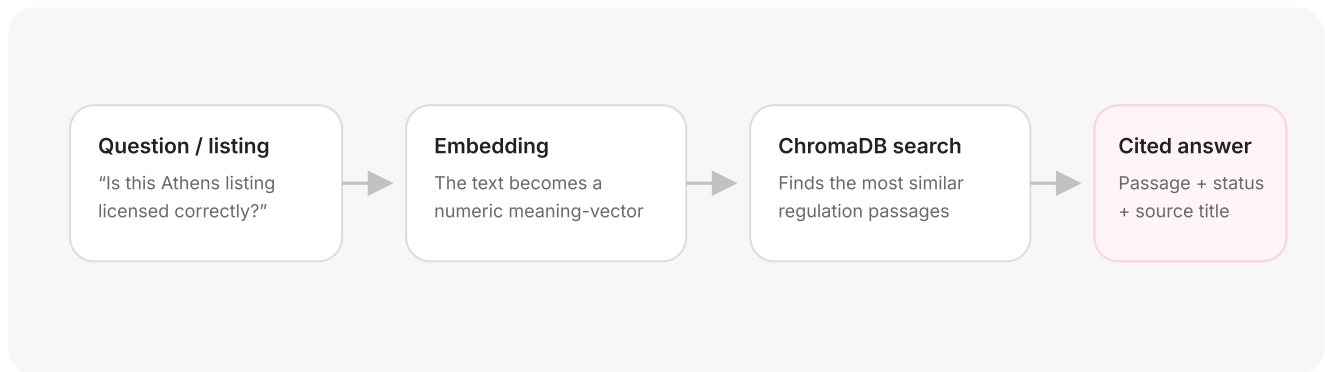
How the live product uses it

When a user asks a forecast-type question (“how will demand develop in Athens?”), the demand agent loads the committed forecast tables, ranks neighbourhoods by forecast strength, shows the 12-month line for the leader, marks its peak month, and renders the region map — with the scenario caveat attached to the answer.

Output files: `prophet_paris_forecast_v1.csv` · `prophet_athens_forecast_v1.csv` (12 monthly rows per neighbourhood), plus the two fitted models in `models/` .

Connecting listings to the law

A market answer is incomplete if the listing itself may be illegal. **RAG (Retrieval-Augmented Generation)** is the technique that fixes this: instead of letting a language model answer regulatory questions from memory (where it can invent things), the system first **retrieves the actual regulation text**, then builds its answer from the retrieved passage — with a citation.



The pieces: an **embedding** converts text into a list of numbers that captures meaning, so “licence” and “registration permit” land close together; **ChromaDB** is a vector database — a search engine over those meaning-vectors — holding the project's regulation corpus for the two cities.

The two regulatory regimes in the corpus

Athens — AMA (Αριθμός Μητρώου Ακινήτου, the property registry number). Greek law requires every short-term rental to display a registration number; operating without one risks fines and platform removal.

Paris — Loi Le Meur. The French framework limiting primary-residence rentals to 90 nights per year, requiring city registration (and a SIRET business identifier in professional cases), with enforcement powers for the city.

The compliance findings — and a deliberate deployment choice

137

Unlicensed Athens listings

Active listings with no AMA registration number in the data

27

Classified high compliance risk

The first queue for manual legal review

≈ €1.03M

Annual revenue at stake

Revenue attached to unlicensed listings — exposed to enforcement

Two readings of the same number

Compliance reading: 137 listings need registration review, prioritised by the triage bands in the committed report. **Investor reading:** if enforcement removes unlicensed supply, roughly €1.03M of annual demand redistributes to compliant neighbours — a quantified supply-shock scenario for the developer persona.

How it is deployed — honestly conservative

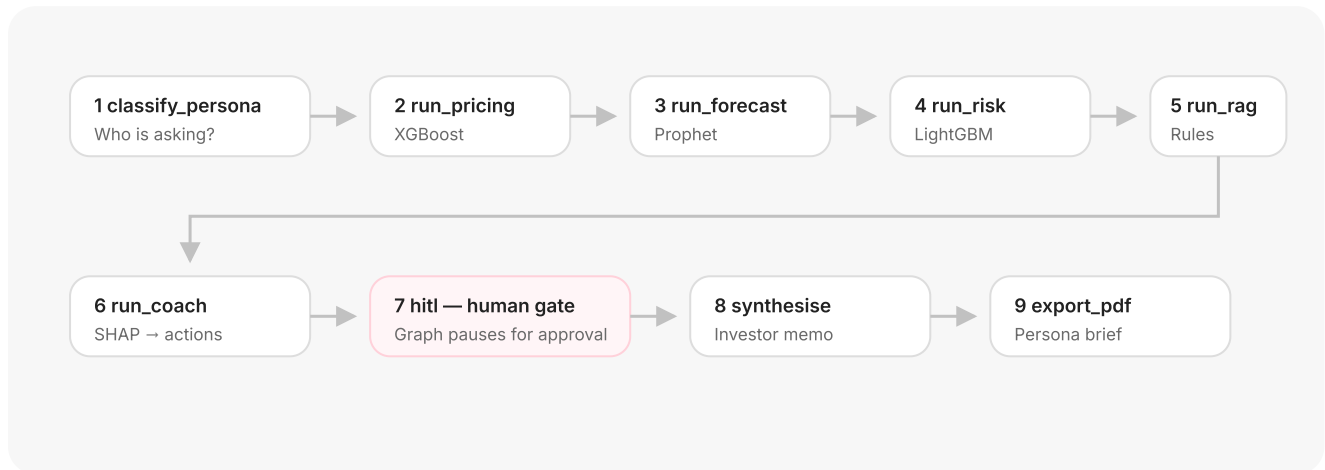
The notebook built the full retrieval pipeline: corpus, ChromaDB index, retrieval utility, and a session log proving the queries. For the live product, the team made a governance decision: the deployed app consumes **committed handoff files** (the unlicensed-listing report, the compliance index with citation titles, and the session log) rather than running live vector retrieval at request time. Compliance answers are therefore **analyst triage grounded in committed evidence — explicitly not legal advice** — and every compliance answer says so. The reproducible ChromaDB utility remains in the repository (`rag/chromadb_retriever.py`) for anyone who wants to rebuild the index locally.

Why this design earns points rather than losing them: a live legal-retrieval system would need counsel-grade guarantees the project cannot make. Serving versioned, reviewable triage files keeps the demo truthful, reproducible, and inside the team's competence — a Trusted AI decision, documented as such.

Output files: `rag_unlicensed_report_v1.csv` · `rag_compliance_index_v1.json` · `aria_rag_session_log.json` · corpus archive `models/rag_corpus_v1.pkl` .

Making five specialists act as one advisor

A real investor question — “should I buy in this Athens neighbourhood?” — needs pricing, demand, risk, and compliance *together*. **LangGraph** is a framework for building exactly that: a directed graph of steps (“nodes”) through which a shared state flows, with memory and controlled interruptions.



The shared state is **ARIAState** — a typed structure of 17 fields carrying the persona, the question, every agent's output, the draft narrative, and the approval status from node to node. **MemorySaver** checkpoints that state at every step, which is what makes the pause-and-resume at node 7 possible.

Human-in-the-loop, evaluation, and what Phase 7 inherits

The human gate (node 7)

HITL (Human-In-The-Loop) means the system cannot finish on its own: the graph is configured to *interrupt before* the synthesis step, so a person reviews the assembled agent outputs and explicitly approves before any memo is generated. This is a direct implementation of KPMG's Trusted AI requirement that consequential outputs carry human accountability — built into the graph's structure, not bolted on.

How the orchestrator was validated

9

Nodes in the executed graph

classify → 5 specialists → human gate → synthesise → export

100%

Routing accuracy · 12 queries

Every test question reached the right persona and specialists

96/96

Phase rubric score

Fully executed notebook with committed session evidence

The evaluation ran **12 persona-labelled queries** (investor, host, developer variants) through the graph and recorded which agents fired: 100% persona classification and 100% specialist recall, with the results committed as `aria_routing_eval.csv` and a full session log. The notebook also generated the five-section investor memo and exported `aria_investor_brief.pdf`.

Phase 6 vs Phase 7 — research layer vs production layer

An important honesty point for the defense: the deployed web product does **not** execute this Python graph at request time. Phase 6 is the **orchestration research layer** — it proves the multi-agent architecture end to end in a checkpointed, interruptible graph. Phase 7 **productionises the same concept** in JavaScript: the live backend mirrors the routing (scope router + agent selection), the grounding (specialist outputs feed the answer), and the governance (quality gates, sources). The concept is identical; the runtime is re-implemented for the web platform. Both layers commit their evidence.

Why not run LangGraph in production? The demo's serverless platform runs JavaScript, and a research notebook is not a hardened service. Re-implementing the pattern natively for the web keeps the live product fast and reliable while the notebook remains the academically validated architecture proof.

A deployed product, not a slide of one

Phase 7 is a live web application — a chat interface in the style of modern artificial intelligence assistants, deployed on Vercel and usable by anyone with the link. It is where every previous phase becomes visible to a non-technical user.

What a user experiences

- **Auto Agent routing** — the default mode. ARIA reads the prompt and selects the right specialist itself; the user never has to know which model answers.
- **Direct-recommendation answers** — every analytical reply opens with the action, then reasoning, key evidence, visuals guidance, limitations, next actions, and sources.
- **KPI (Key Performance Indicator) cards** — headline numbers with plain-language help text explaining what each metric means and whether high or low is better.
- **Charts and region maps** — rankings, comparisons, distributions, and interactive neighbourhood maps that render by default whenever a question is geographic.
- **Follow-up awareness** — “what about the risk there?” keeps the city and context from the previous turn.
- **Exports** — a Chat Brief (a formatted document of the full conversation) and a Project Brief (a reusable analyst instruction in Markdown, grounded in the conversation).
- **Eight scripted demo prompts** — curated flows for reliable presentation, one per persona story.

The five agents a user can meet

AGENT	WHAT IT ANSWERS
Market Entry Advisor	Neighbourhood ranking, opportunity scores, entry timing — the investor's default.
Host Revenue Intelligence	Underpricing gaps, listing health, concrete revenue actions.
Tourism Demand Forecast	Occupancy signals and the 12-month scenario demand outlook.
Gentrification Early Warning	Saturation and displacement-pressure signals by area.
STR Financial Crime Detection	Compliance triage and anomaly-style risk framing (STR = Short-Term Rental).

Under the hood of a live answer

The backend is the five-step pipeline from the architecture (section 04). Three engineering details are worth understanding because they define the product's character:

1 · Scope routing with graceful boundaries

A lightweight model (Gemini 2.5 Flash) classifies every prompt: Paris/Athens rental analysis runs the full analytics path; general questions ("what time is it in Madrid?") get a normal assistant answer with live web search; questions about other cities get an honest "ARIA is not trained on that market" plus the nearest in-scope offer. The router reads recent conversation, so short follow-ups stay in their lane.

2 · Deterministic grounding before generation

The analytics pipeline fetches the committed output files (20-minute cache), computes the KPIs, rankings, maps, and charts for the detected intent, and assembles a verified analytics pack. Only then is the main model (Gemini 2.5 Pro by default; Anthropic's Claude selectable) asked to write the explanation *around* that pack. The model adds language, never facts.

3 · Quality gates with a deterministic fallback

Before an answer ships, the backend checks structure (recommendation first, usable sections, no invented cross-city pivots), completes truncated sentences, humanises raw column names, verifies the mandatory sources line, and — if the model's draft fails the bar — substitutes a deterministic template built from the same analytics pack. Representative answers are regression-tested and expected to score at least 97/100 on the project's response-quality evaluation.

5**Backend steps per question**

Route · resolve context · ground ·
generate · assemble

20 min**Evidence cache lifetime**

Committed outputs re-fetched from
GitHub raw at most every 20 minutes

≥ 97/100**Response-quality bar**

Regression-tested on representative
prompts before release

Model transparency: if a selected model is unavailable on the cloud platform, the app falls back to the default model and says so in a visible notice — the user is never silently switched.

The Athens story: three pools of money

Read as one narrative, the models describe three distinct, quantified opportunity pools in Athens — each with a different owner and a different action.

€4.8M / yr

Pool 1 — Underpricing

2,945 listings priced > €15 below fair price · owner: hosts & managers

€1.43M / yr

Pool 2 — Priority overlap

865 listings underpriced AND high-risk · owner: portfolio operators

≈ €1.03M / yr

Pool 3 — Compliance exposure

137 unlicensed listings · owner: regulators & compliant competitors

Pool 1 — close the pricing gap

For hosts: recalculate fair price, review the SHAP drivers, and move price in controlled steps — prioritising the high-confidence tier (gaps above €25–€40). The €4.8M aggregate is the market-wide ceiling, not a per-host promise.

Pool 2 — stabilise, then reprice

The 865 overlap listings are urgent but fragile: raising price on a listing with collapsing demand can accelerate decline. The sequenced playbook — fix quality signals and booking momentum first, then close the gap — is exactly what the listing coach generates per listing. Conservative case if only half of each gap is captured: **€0.71M per year**.

Pool 3 — the supply-shock scenario

If enforcement removes the 137 unlicensed listings, their ≈ €1.03M of annual demand redistributes to compliant neighbours. For the developer persona this is a quantified entry thesis; for a host it is a reason to verify licensing before competing on price.

Where to start on the map

Zappeio (ΖΑΠΠΕΙΟ) leads the neighbourhood ranking with a 0.955 opportunity score and ≈ €15,416 estimated annual revenue. The wider pattern: the centre commands a 3.9× revenue premium over the periphery, but the “near” ring offers the best gross yield (2.4%) — strong revenue at materially lower entry prices.

Recommendations by persona

For the institutional investor

- Shortlist from the opportunity ranking (Zappeio first), then stress-test each candidate against saturation and the risk map before any site visit.
- Prefer the "near" ring where yield (2.4%) beats the postcard centre (2.3%) at lower entry prices; treat "mid" (1.8%) as the value trap the data says it is.
- Use the demand scenario forecasts to time entry against seasonality — and treat them as a screen, not a booking guarantee.

For the host / property manager

- Check your listing against the underpricing file; if your gap exceeds €25, the model considers the signal high-confidence.
- Read your SHAP drivers before touching price: if the model's discount comes from a weak review score or missing amenities, fix those first — that is the coach's sequenced playbook.
- If you are in the 865-listing overlap, act this season: you are losing revenue while your demand signals decline.

For the developer / private-equity fund

- Model the supply shock: 137 unlicensed listings $\approx$ €1.03M of redistributable annual demand, concentrated where the compliance map shows high-risk density.
- Compare markets honestly: Paris offers scale (120,809 listings of evidence), Athens offers current, complete signals — the capstone's proof-of-concept market.
- Use the portfolio comparison view for the cross-city entry decision, then neighbourhood ranking within the chosen city.

The boundary on every recommendation: ARIA is decision support, not decision replacement. Its outputs shortlist and prioritise; final investment, pricing, and legal decisions still require property-level due diligence, financing analysis, and — for compliance — qualified counsel. The product says this about itself, on purpose.

Five habits that make ARIA defensible

“Trusted AI” is KPMG's framework for artificial intelligence that organisations can actually rely on: explainable, honest about uncertainty, supervised by humans, and quality-controlled. ARIA implements it as engineering habits, not slideware.

1 · Explainability everywhere

Every model ships with SHAP (Shapley Additive Explanations) outputs; every live answer carries its reasoning, its metric definitions, and its exact source files. Nothing asks to be trusted blindly.

2 · The leakage correction, celebrated not hidden

A suspiciously perfect risk score (0.9995) was investigated with the model's own explanation tools, diagnosed as data leakage, corrected, and re-reported honestly at 0.8288 — with the whole story documented in the notebook and the defense materials.

3 · Human-in-the-loop by construction

The orchestration graph physically pauses before synthesis until a person approves (the `interrupt_before` gate). Consequential outputs carry human accountability as an architectural property.

4 · Contract-validated evidence

Every committed output file is checked in CI (Continuous Integration) against `schemas/output_contracts.json` — schema, row counts, and numeric ranges — by `scripts/validate_outputs.py` on every push. The live product cannot silently consume malformed evidence.

5 · Scope honesty at the product edge

The live app declines to fabricate: other-city questions get a data-boundary answer, compliance answers are labelled analyst triage rather than legal advice, scenario forecasts carry their caveat, and model fallbacks are disclosed in the interface.

If one sentence must carry this section: ARIA treats honesty as a feature — the project's strongest defense moments are the places where it disclosed a weakness and engineered around it.

What the numbers can and cannot support

Every limitation below is disclosed in the repository. Knowing them is what lets the team present confidently — none of these is a surprise waiting to happen.

Paris data vintage — the structural caveat

Paris combines 2021 observed prices (Maven) with 2025 behavioural data (Inside Airbnb) whose price field was partly estimated. Consequences: the Paris pricing model (R^2 0.588) is presented as a cross-market comparison tool, estimated prices never enter training, and current-market claims belong to Athens. Any slide that quotes Paris pricing should carry this footnote.

The engineered risk label is a proxy

No public dataset records true host churn. `at_risk_host` is built from six observable warning signs and validated for transparency, not ground truth. It supports *prioritisation* (who to look at first) — it does not support statements like “this host will fail”. The continuous probability and adjustable 0.70 threshold exist precisely so users can set their own strictness.

Scenario forecasts, not crystal balls

The Prophet outputs derive from annual occupancy estimates, assumed seasonality, and capped review momentum — not observed booking calendars. The notebook records that prediction intervals fell below target coverage. Correct use: comparing neighbourhoods' relative demand shape. Incorrect use: promising anyone a specific month's occupancy.

Model error bounds every euro figure

With a €29.1 mean absolute error, individual small gaps are hints; the €4.8M, €1.43M, and €1.03M figures are aggregate opportunity estimates whose per-listing components vary in confidence. The €15 threshold nominates candidates; €25–€40 gaps carry conviction.

Scope boundaries and how they are managed

Language bias in sentiment

VADER (the sentiment scorer) reads French less reliably than English, inflating Paris scores by about +0.178. The project's response: a comparability flag, and a standing rule that cross-city sentiment comparisons are invalid. Sentiment is used within-city only.

Two cities, one framework

ARIA's models are trained for Paris and Athens specifically — separate models per city because the markets differ structurally. The framework generalises (the pipeline would retrain on a new market's data); the current numbers do not. The live product enforces this honestly by declining data-backed analysis of other cities.

The live product serves committed outputs, not live retraining

Answers are grounded in versioned files produced by the validated notebooks. This is a deliberate reliability choice for a capstone demo — but it means the data plane updates when notebooks re-run and commit, not continuously. A production roadmap would add scheduled retraining and monitoring.

Compliance triage is not legal advice

The RAG (Retrieval-Augmented Generation) layer retrieves and cites regulation text and triages exposure. Regulatory interpretation for a real transaction requires qualified counsel — the product's own answers say so.

Residual risks worth monitoring

- **Neighbourhood benchmark hygiene:** `neighbourhood_median_price` is a legitimate market-context feature, but a production version should compute it from training-only or historical windows to remove any residual evaluation contamination.
- **Data drift:** September 2025 signals age; quarterly refreshes would keep the Athens story current.
- **Regulation changes:** the compliance corpus reflects the rules as collected; legal frameworks move and the corpus needs versioned updates.

Data & methodology questions

The safest answer structure: business reason → data reason → modelling reason → honest limitation → what we would improve next.

Q. Why combine four data sources instead of using one clean dataset?

A. No single public source had prices, locations, reviews, availability, licences, and neighbourhood context together. Each source contributed a piece; routing flags manage the quality differences so each model only sees rows appropriate for its task.

Q. Why is Athens the hero if it is only 14,242 of 135,051 rows?

A. Athens is smaller but richer: current September 2025 data with observed prices, licences, reviews, and calendar-derived signals. It is the only city where pricing, risk, and compliance connect end-to-end today. Paris provides scale and proves the framework travels.

Q. Why predict log-price instead of raw price?

A. Rental prices are heavily skewed; a few luxury outliers would dominate the error. Modelling the logarithm compresses extremes so the model learns the whole market — predictions are converted back to euros for interpretation.

Q. Why two separate city pricing models?

A. The exploratory analysis measured an Athens distance effect 2.8× stronger than Paris, and SHAP later confirmed different drivers — Paris size-and-capacity, Athens quality-location-demand. One combined model would blur both markets.

Q. Is the €15 underpricing threshold defensible?

A. As a discovery threshold, yes: it is meaningful against Athens nightly prices and deliberately below the €29.1 average model error, so it over-collects candidates rather than missing them. Confidence tiers (€25–€40+) and SHAP explanations do the filtering. It is a candidate-finder, not an automatic pricing command.

Q. Could the neighbourhood median price feature leak the answer?

A. It is a market benchmark, not the listing's own price — the model never sees price_eur or log_price. It is transparent and interpretable; in production we would compute it from training-only or historical windows to remove any residual contamination risk.

Model choice & validation questions

Q. Why XGBoost rather than linear regression or a neural network?

A. Pricing is nonlinear and interaction-heavy — a central two-bedroom behaves nothing like a distant private room. Gradient-boosted trees capture that on tabular data without manual specification. Linear regression is too simple; a neural network would be harder to explain and tune for no expected gain on this data size — and explainability is a KPMG requirement.

Q. Why LightGBM for risk?

A. A different task: classification with a probability output. LightGBM is fast, handles the 56.8/43.2 class imbalance natively, and pairs with SHAP for per-listing explanation.

Q. What exactly is data leakage, and what happened here?

A. Leakage is when training data contains information that reveals the answer. Our first risk model scored 0.9995 because one feature was computed from the same availability signal used in the label. SHAP exposed it, we removed the leaking columns, and the honest score is 0.8288. We present this as our best governance moment — we did not accept a fake-perfect model.

Q. Is 0.8288 AUC good enough?

A. Yes, for an early-warning ranking task on public data with an engineered label: far above the naive baseline, stable across folds (± 0.0088), calibrated (Brier 0.1656), and used for prioritisation — never for automatic irreversible decisions.

Q. Why AUC instead of accuracy?

A. With 56.8% of listings at-risk, predicting “risky” for everyone scores 56.8% accuracy while learning nothing. AUC measures whether the model ranks risky above safe across all thresholds — the property a prioritisation tool actually needs.

Q. Why is the engineered at_risk_host label valid science?

A. Public data has no true churn outcome, so we built a transparent proxy from six observable warning signs, disclosed it, and output probabilities rather than verdicts so users choose their own strictness. Imperfect, explainable, useful — in that order.

Product & challenge questions

Q. Why not just use ChatGPT or Gemini for everything?

A. A language model can explain, but it should not invent pricing or risk scores. In ARIA the numbers come from validated, versioned model outputs computed before the language model is called; the language model writes business language around them. That separation — analytics compute, language explains — is the whole architecture.

Q. Is the deployed app really connected to the models?

A. Yes — to their committed outputs. The live backend fetches the versioned prediction, risk, forecast, and compliance files, computes every KPI and visual from them, and grounds each generated answer in that pack. It does not retrain models at request time, and the orchestration research graph (Phase 6) runs in the notebook rather than in the web runtime — both disclosed design choices, not gaps.

Q. Should I trust models trained on imperfect public data?

A. Trust the discipline, not the data: limitations documented, routing flags protecting training, estimated prices excluded, leakage corrected, uncertainty surfaced. The project is a defensible proof of concept for explainable advisory workflows.

Q. Are the 2,945 underpriced listings definitely underpriced?

A. They are model-identified candidates. With €29.1 average error, small gaps are hints; the €25 median gap sits at the confidence boundary and larger gaps carry conviction. ARIA presents gap size, confidence tier, and drivers together so users act proportionately.

Q. Why does the demo include scripted prompts?

A. Presentation reliability. The eight scripted flows guarantee the audience sees the intended stories with stable numbers; the same interface accepts free-form questions through the full live pipeline. Scripted and live modes are both real — one is rehearsed, the other is not.

Q. What would you build next?

A. Scheduled data refresh and drift monitoring; per-listing SHAP drivers in the risk export; neighbourhood benchmarks computed on training-only windows; live retrieval behind the compliance triage with counsel review; and a third market to prove the retraining playbook.

Every technical term, A–L

Agent

A software component with one specialised job (for example, pricing) that a larger system can call on.

AMA (Αριθμός Μητρώου Ακινήτου)

Greece's property registry number — the licence a short-term rental must display.

API (Application Programming Interface)

A defined way for one piece of software to request work from another.

AUC-ROC (Area Under the Receiver Operating Characteristic Curve)

A 0-to-1 ranking-quality score for classifiers; 0.5 is guessing, 1.0 is perfect ordering.

Brier score

Measures whether predicted probabilities are calibrated — whether “70% risky” behaves like 70%. Lower is better.

ChromaDB

A vector database — a search engine over meaning-vectors — used for regulation retrieval.

CI (Continuous Integration)

Automated checks that run on every code push; here they validate the committed output files.

Cross-validation

Re-training and re-testing a model on several data splits to prove results are stable, not lucky.

Data leakage

When training data accidentally contains the answer, inflating scores. Found and fixed in Phase 3.

EDA (Exploratory Data Analysis)

Systematic inspection of a dataset before modelling — quality, patterns, and design decisions.

Embedding

A numeric representation of text that captures meaning, so similar ideas land close together.

Gross yield

Annual rental revenue divided by property price, before costs — a quick comparability measure.

Haversine formula

Computes the distance between two latitude/longitude points along the Earth's surface.

HITL (Human-In-The-Loop)

A design where the system pauses for human approval before finishing consequential work.

Holdout set

Data locked away during training and used only for the final honest evaluation (20% here).

KPI (Key Performance Indicator)

A headline number chosen to summarise performance — shown as cards in the live product.

LangGraph

A framework for wiring specialist steps into a directed graph with shared state, memory, and interrupts.

LightGBM (Light Gradient Boosting Machine)

A fast tree-based machine learning method; here, the Athens risk classifier.

Listing coach

The agent that turns model explanations into concrete host actions (“raise price €X, improve Y”).

LLM (Large Language Model)

An artificial intelligence model trained on text to understand and generate language.

log_price

The logarithm of price — the transformed target that stabilises training on skewed prices.

Loi Le Meur

The French law limiting primary-residence short-term rentals to 90 nights/year with registration.

Every technical term, M–Z

MAE (Mean Absolute Error)

The average size of prediction mistakes, in the target's own units — €29.1/night here.

MemorySaver

LangGraph's checkpointing component — saves the shared state at every node so runs can pause and resume.

Notebook (Jupyter)

An interactive document mixing code, results, and narrative — the project's research format.

Optuna

An automated hyperparameter-tuning tool; it searched 100 configurations for each model.

Opportunity score

ARIA's 0-to-1 composite of revenue upside and market pressure per neighbourhood; Zappeio scores 0.955.

Prophet

A time-series forecasting method built for seasonal business data; produces the 12-month demand scenarios.

Proxy label

An engineered stand-in for an outcome the data does not record directly — like `at_risk_host`.

R² (R-squared, coefficient of determination)

Share of variation a model explains, 0 to 1. Athens pricing: 0.676.

RAG (Retrieval-Augmented Generation)

Retrieving real source text first, then generating the answer from it — with citations.

React / Vite

The framework and build tool behind the live web interface.

Routing flag

A column marking which rows are safe for which task — the dataset's quality guardrails.

Saturation score

ARIA's 0-to-1 measure of how crowded/competitive an area's rental market looks. Lower is calmer.

Scope router

The backend step deciding whether a prompt is in ARIA's domain, general, or another city.

SHAP (Shapley Additive Explanations)

Assigns each feature a signed contribution to each prediction — the explainability backbone.

SIRET

France's business establishment identifier, required in professional rental registrations.

StateGraph

LangGraph's directed-graph structure; ARIA's has 9 nodes sharing a 17-field state.

STR (Short-Term Rental)

The Airbnb-style rental market ARIA analyses.

VADER (Valence Aware Dictionary and sEntiment Reasoner)

A rule-based sentiment scorer used on 594,482 reviews; weaker on French text (+0.178 Paris bias).

Vercel

The cloud platform hosting the live product and its serverless backend.

Vertex AI

Google Cloud's model platform; runs Gemini 2.5 Pro (default) and hosts Claude as an option.

XGBoost (Extreme Gradient Boosting)

A tree-based method that builds trees sequentially, each correcting the last — the pricing engine.

Where everything lives

LOCATION	CONTENTS
<code>eda/</code>	The six analytical notebooks, Phases 1–6, plus 40 generated figures.
<code>data/outputs/</code>	Every committed model output — predictions, underpricing, SHAP, risk scores, forecasts, RAG handoff, routing evidence.
<code>models/</code>	Trained model artifacts and the model card (governance record).
<code>rag/</code>	The reproducible ChromaDB retrieval utility for the compliance corpus.
<code>Stage 7 - UI Interface/vercel_vite_app/</code>	The live product — React frontend and the five-step backend.
<code>schemas/ & scripts/</code>	Output contracts and the validation script that CI runs on every push.
<code>docs/</code>	Project summary, mentor preparation documents, methodology reviews.
<code>README.md & agent.md</code>	The authoritative project record and the agent handoff memory.

Verify it yourself, in two commands

`python scripts/validate_outputs.py` checks every committed evidence file against its contract. The live demo runs at the Vercel link in the README. Everything this guide claims traces to one of those two places.

ARIA — Agentic Real-estate Intelligence Advisor.

IE University Sci-Tech School × KPMG Spain · Corporate Capstone 2026.

Data: Maven Analytics (2021) · Inside Airbnb (September 2025) · Gyodi & Nawaro (2021).

This guide was generated from the repository main branch on 3 July 2026. Architecture diagram designed in Figma using the Airbnb design language (Rausch #FF385C · Inter as the documented Airbnb Cereal substitute).